

Data Structures – Unit 11

Bucharest University of Economic Studies

Balanced Trees



Agenda

- Definition
- Characteristics
- Complexity
- Representation
- Implementation
- Operations



AVL - Definition

A binary search tree, BST, which implements a equilibrium condition is called a self-balanced binary search tree, AVL (Georgy **A**delson-**V**elsky and Evgenii **L**andis).



AVL - Characteristics

- The balancing condition is based on the equilibrium degree of each subtree extracted from the original tree.
- In an AVL tree, the heights of the two child subtrees of any node differ by at most one unit.



AVL - Characteristics

The balancing condition for each subtree is:

$$GE = H(LS) - H(RS),$$
$$GE \in \{-1, 0, 1\}$$

where:

$H(LS)$ – the height of the left subtree of a node;

$H(RS)$ – the height of the right subtree of a node;



AVL - Characteristics

The height of a node:

```
int H(AVL* root)
{
    return (root==NULL)? 0 : 1 +
           max(H(root->left),H(root->right));
}
```

!!! The height of a void subtree is 0 !!!



AVL - Complexity

The lookup, insertion and deletion operations take $O(\log n)$ time both in worst and best cases scenarios.

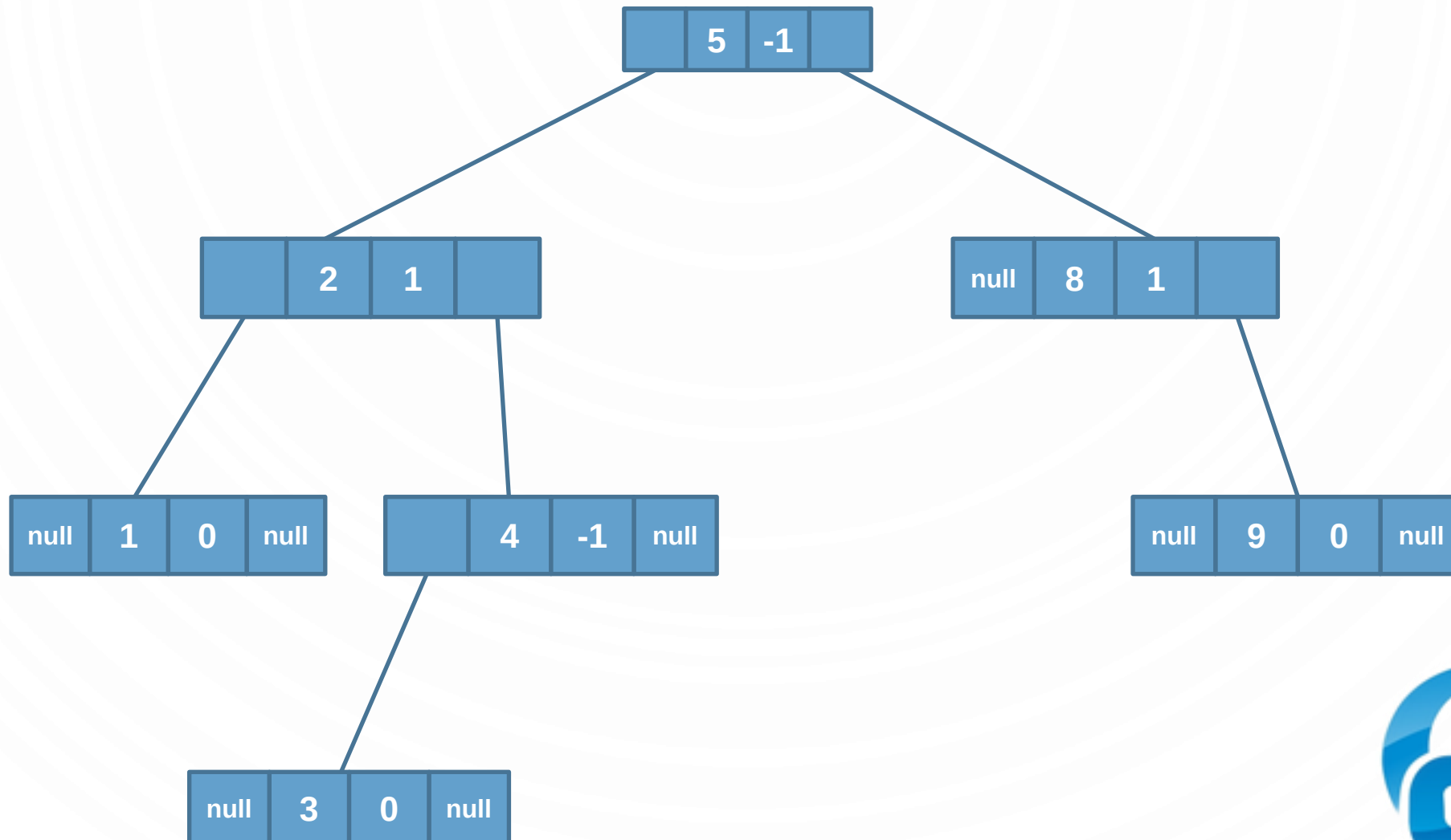


AVL – Data Structure

```
template<class Type> struct AVLNode{  
    Type info;  
    AVLNode<Type>* left;  
    AVLNode<Type>* right;  
    int eqDegree;  
};
```



AVL – Representation



AVL - Implementation

- Divide et impera;
- Traversing the ordered set of keys same as for a binary search tree;
- Needs rebalancing after each insertion and deletion operation;



AVL - Implementation

Balancing method:

- A self balanced tree must preserve its state after each alteration of its structure;
- Rebalancing must be applied over the node which doesn't respect the balance condition;
- All the nodes from the path affected by an insertion or deletion operation must be checked for balancing condition;



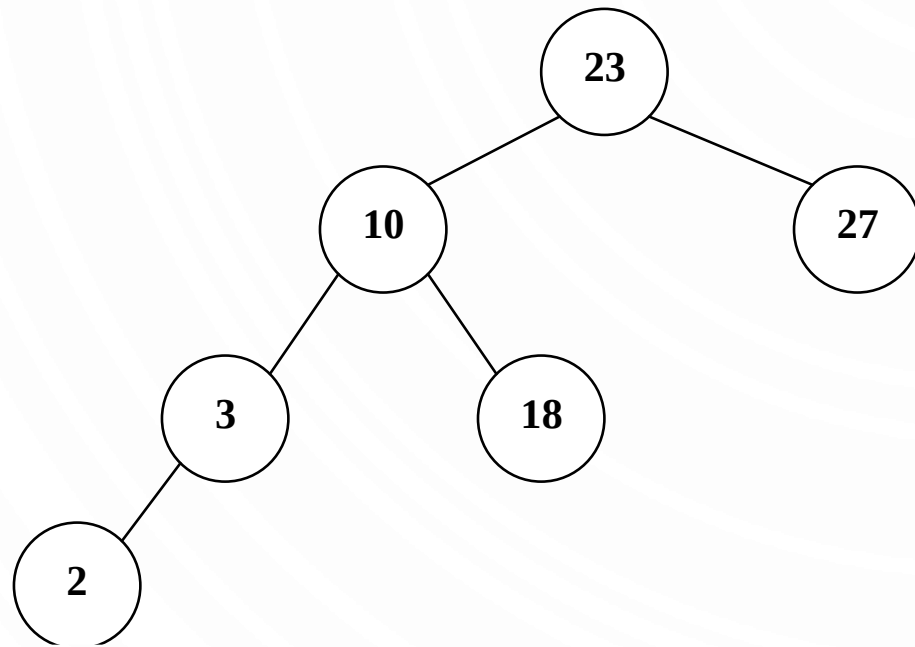
AVL - Implementation

- a $O(\log n)$ complexity time for operations;
- balancing operations for specific balanced trees such as: AVL, B trees, Red and Black trees;

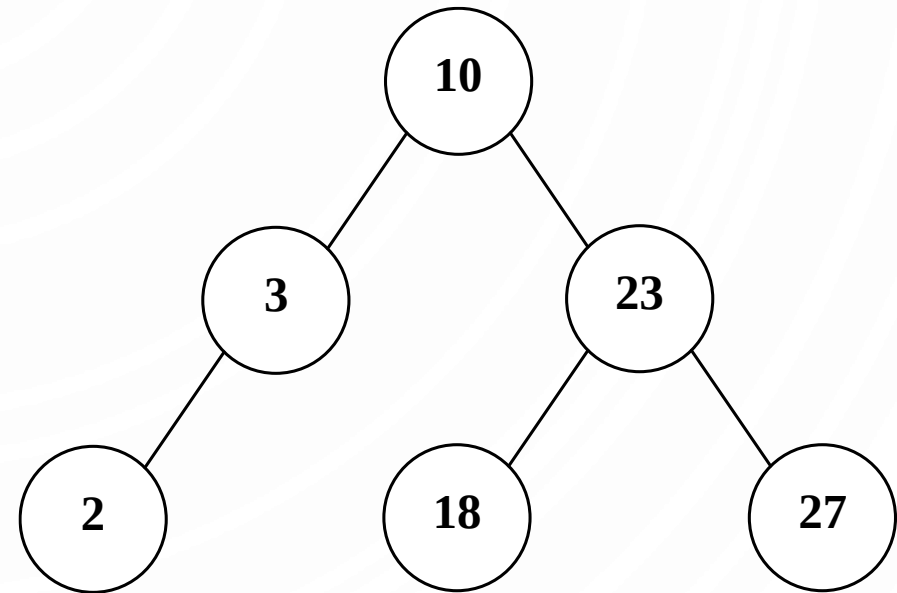


AVL - Operations

Minimizing the lookup time:



23 – 10 – 3 – 2, 4 comparisons

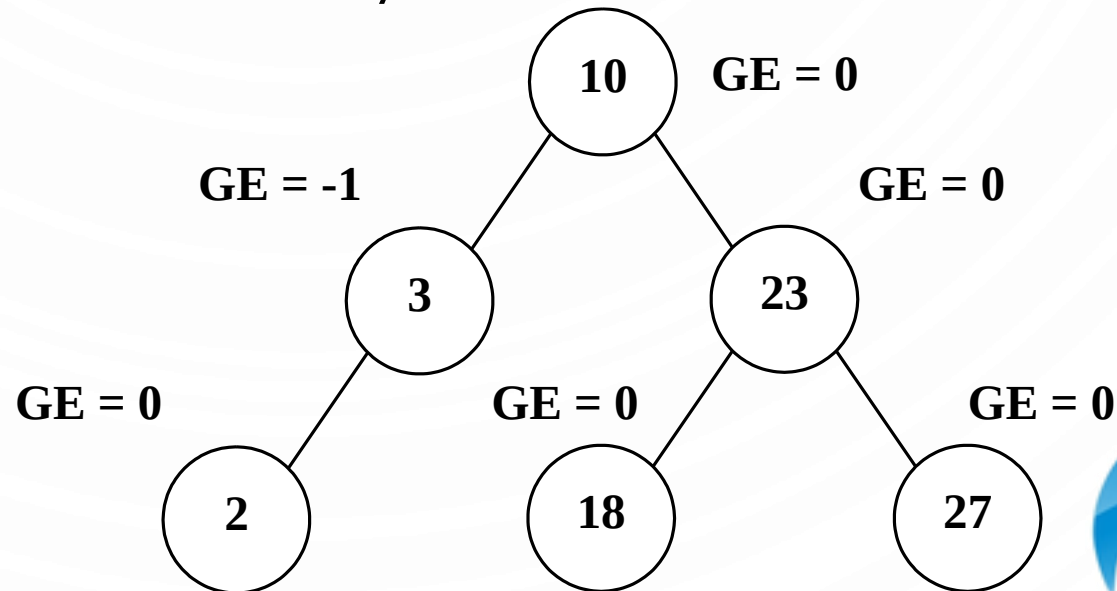


10 – 3 – 2, 3 comparisons



AVL - Operations

- $GE = 0$, balanced node;
- $GE = 1$ or $GE = -1$, node with a minor unbalanced in the right or left subtree;
- a perfect balanced tree has always an odd number of elements;



AVL - Operations

Maintaining a tree AVL structure:

- Check the equilibrium degree for each node;
- Evaluating each insertion and deletion operation not to unbalance the tree;
- Major unbalanced scenarios are found when equilibrium degree of a node has the following two possible values $\{2; -2\}$;



AVL - Operations

Rebalancing of a binary tree in order to keep the structure of a AVL tree:

1. Simple rotation to the left;
2. Simple rotation to the right;



AVL - Operations

- **Insertion**: a tree that get unbalanced as a result of an insertion operation must be balanced before applying one or two rotations depending on the type of the imbalance;
- **Deletion**: much more complex than insertion because this operation can remove nodes not only from the leaves level;

In order to determine which type of rebalance operation needs to be applied, the equilibrium degree of the nodes affected by the operation must be studied.



AVL - Operations

1. Simple rotation to the right;

- Insertion into the left subtree at the left child level;

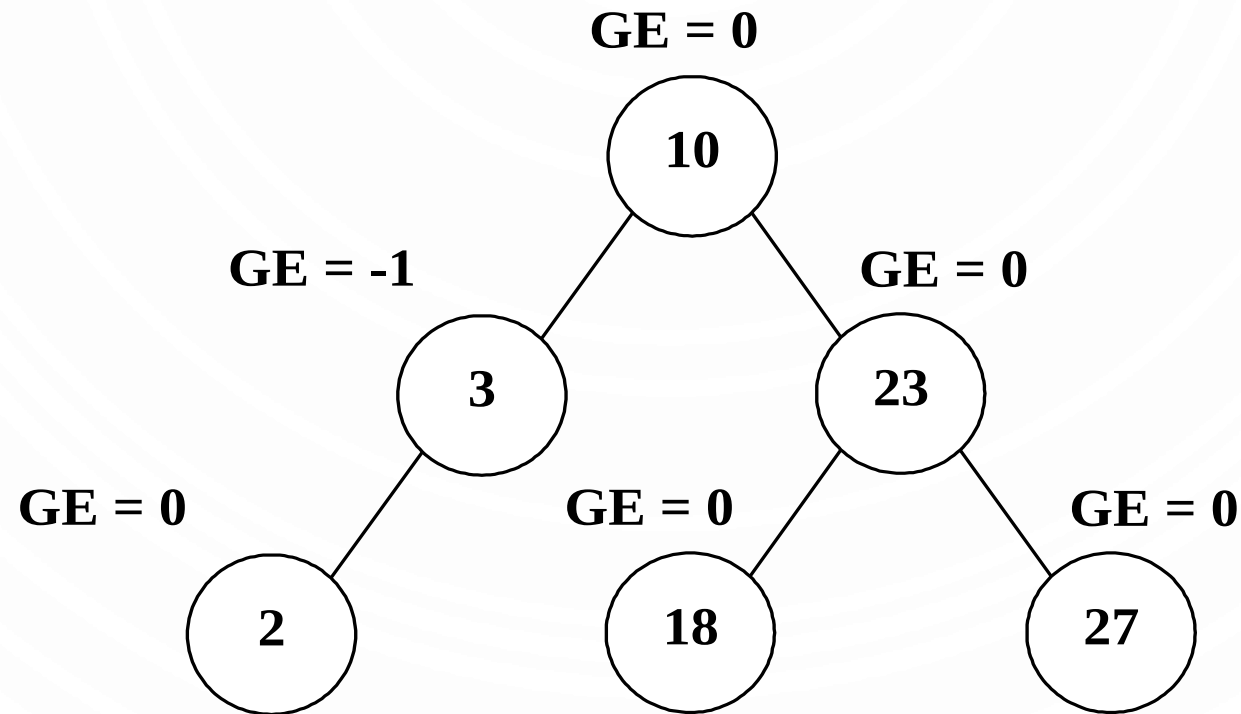
2. Simple rotation to the left;

- Insertion into the right subtree at the right child level;



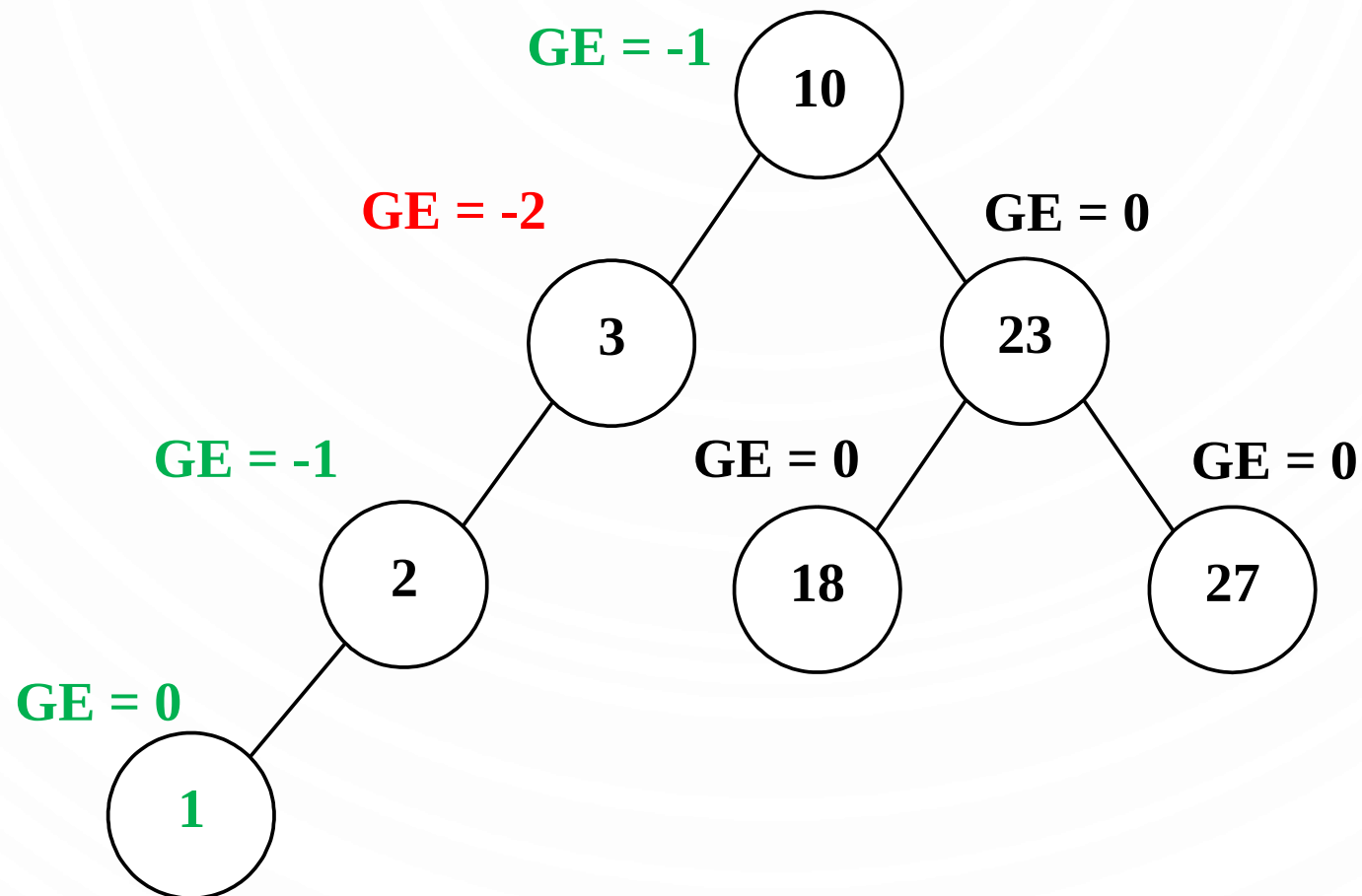
AVL - Operations

Unbalanced AVL tree after inserting the key with value 1;



AVL - Operations

Unbalanced AVL tree after inserting the key with value 1;



AVL - Operations

Rebalancing operations:

- A pivot node is identified; the pivot is the first node from the operation's path in which the GE is equal to 2 or -2;
- **Rebalancing**: as close to the node that triggered the unbalanced condition;
- **Identifying**: the type of rotation based on the equilibrium degree of the pivot and the affected node;

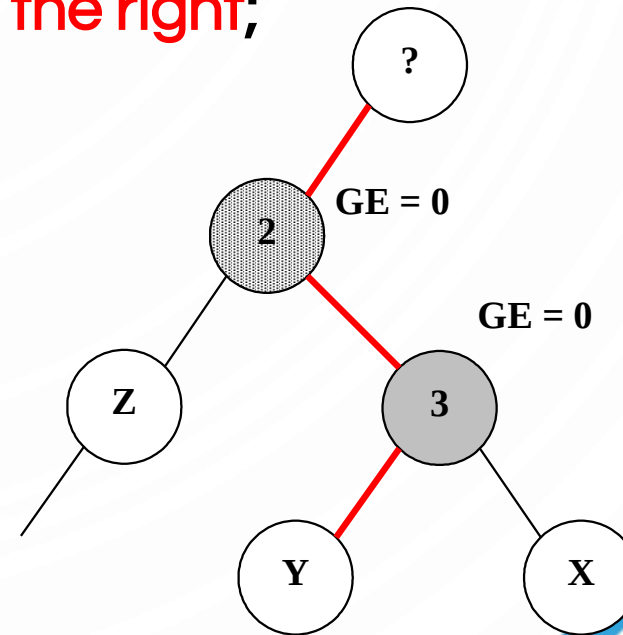
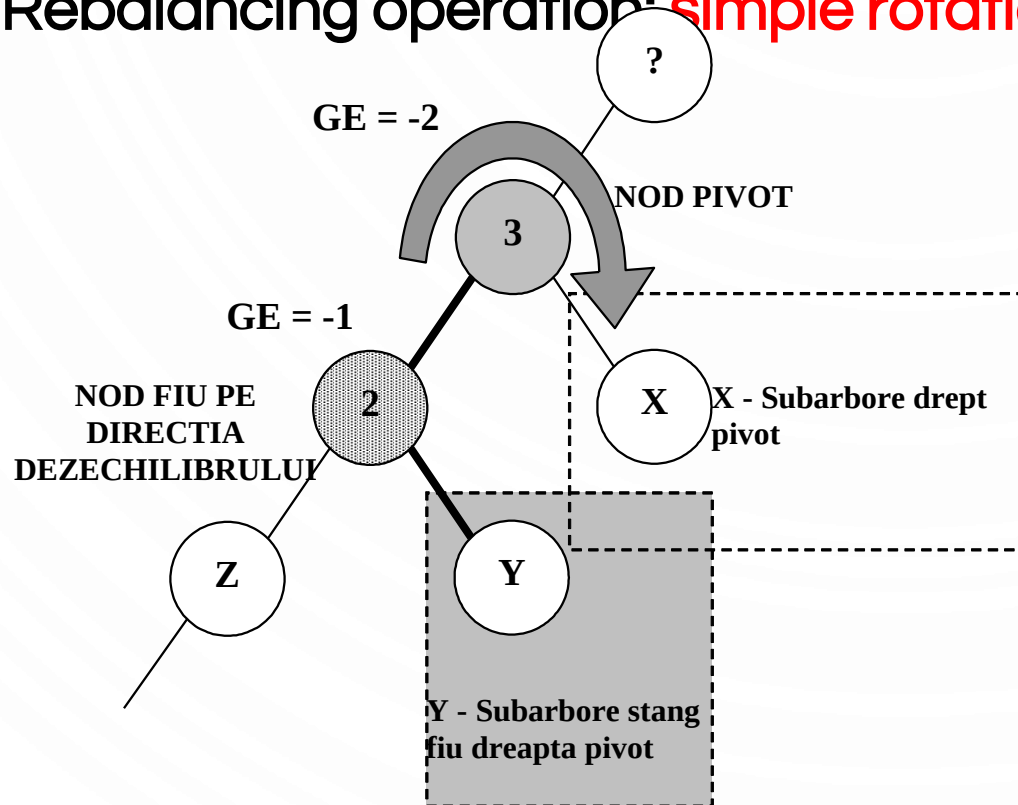


AVL - Operations

Pivot (node 3), has $GE = -2$: unbalanced to the left;

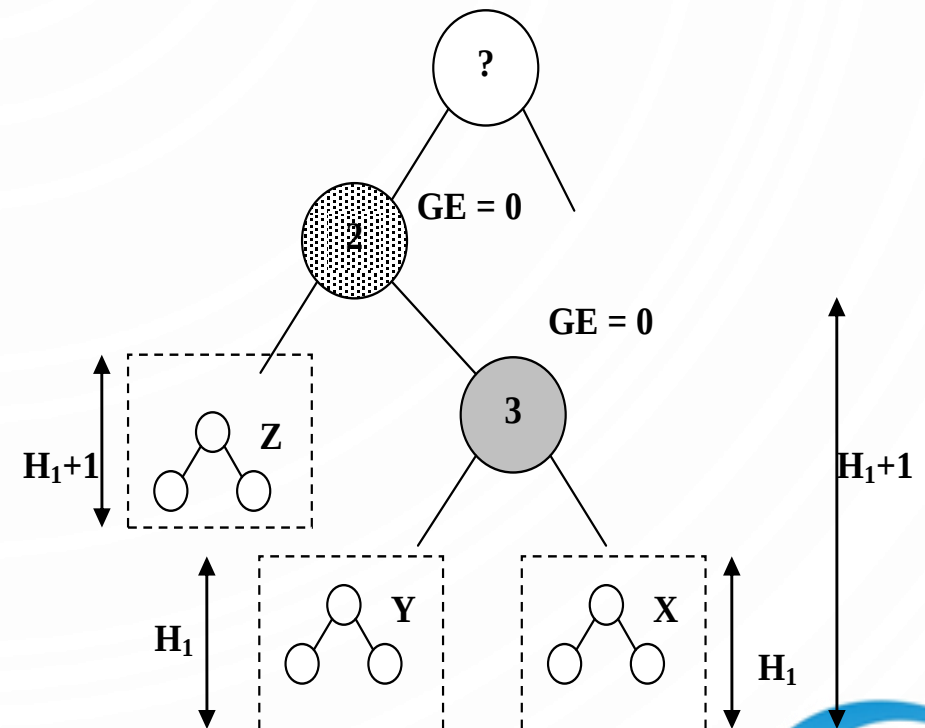
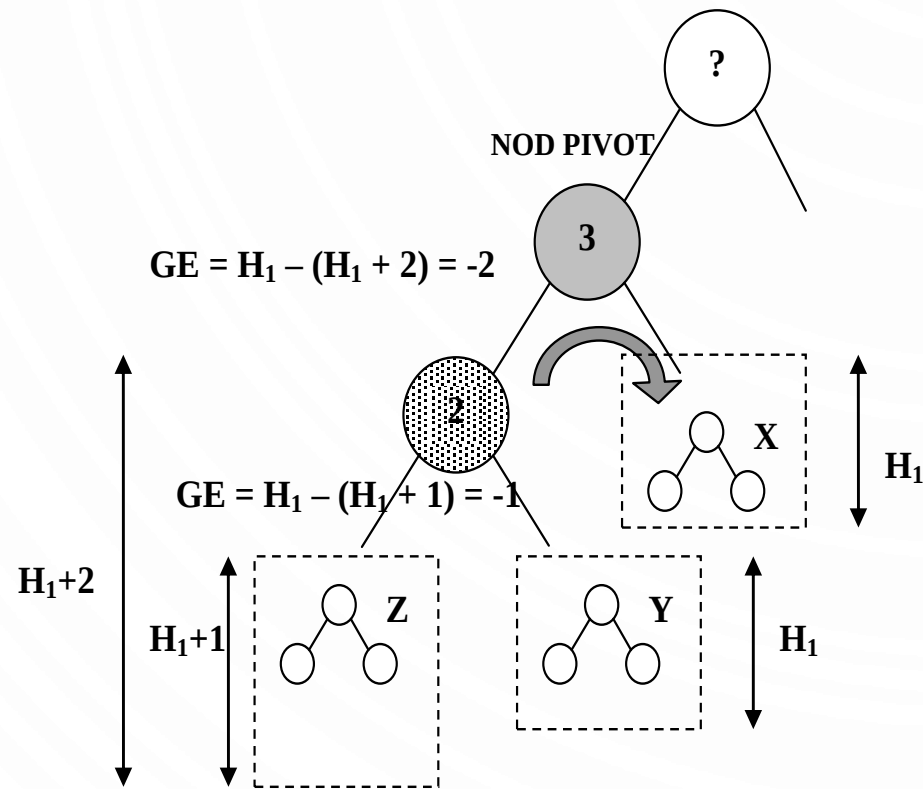
The left child has a minor unbalance to the left as well;

Rebalancing operation: **simple rotation to the right;**



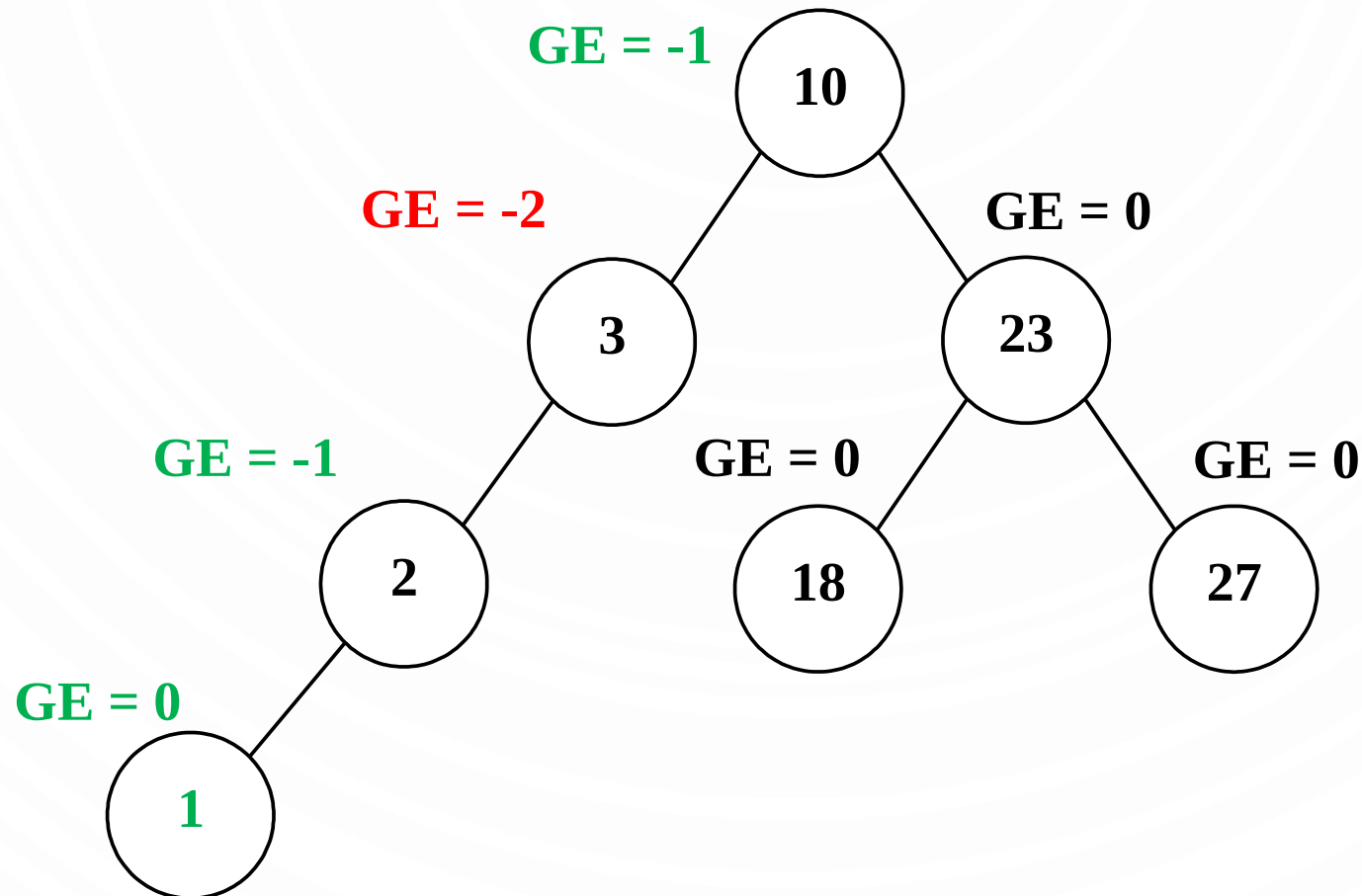
AVL - Operations

Simple rotation to the right



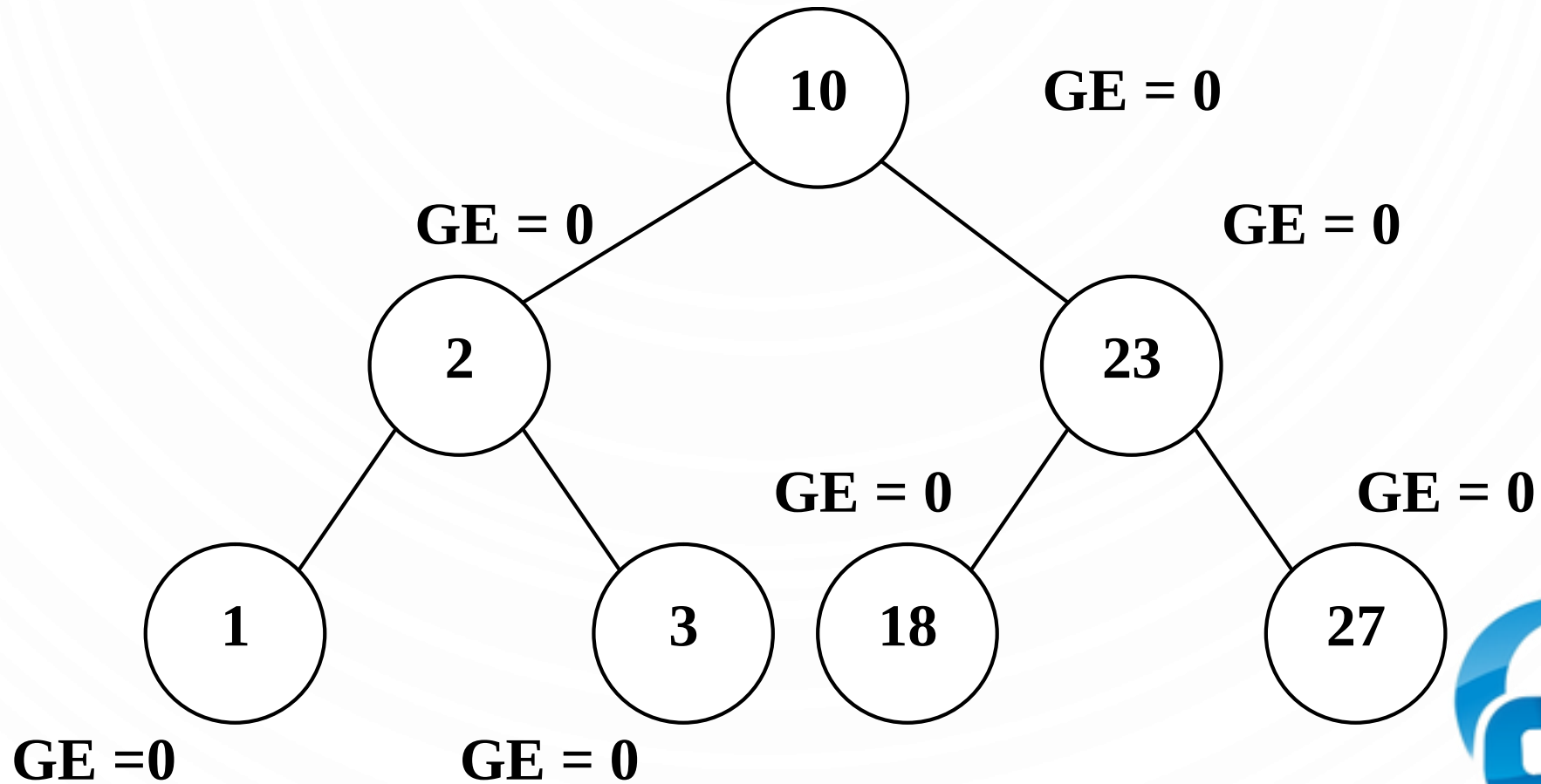
AVL - Operations

Unbalanced AVL tree after inserting the key with value 1;



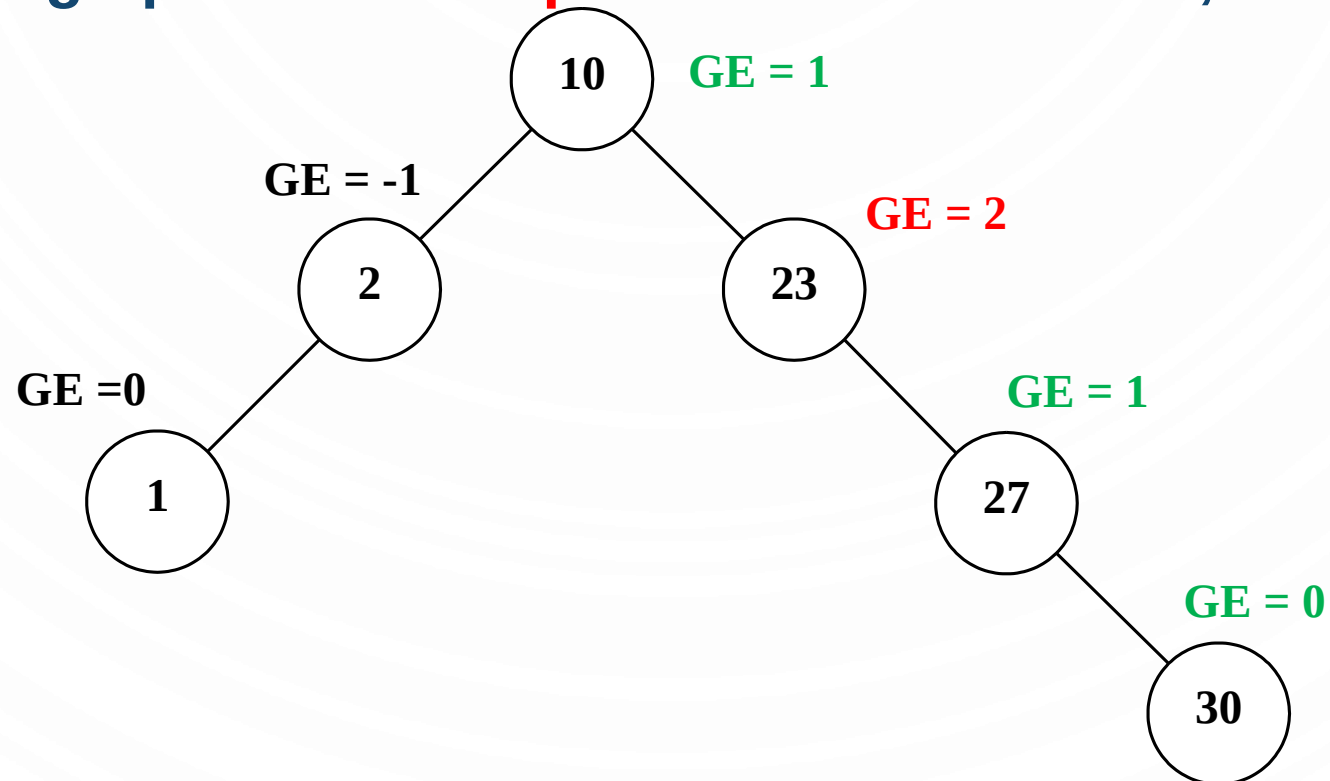
AVL - Operations

Perfectly balanced AVL tree



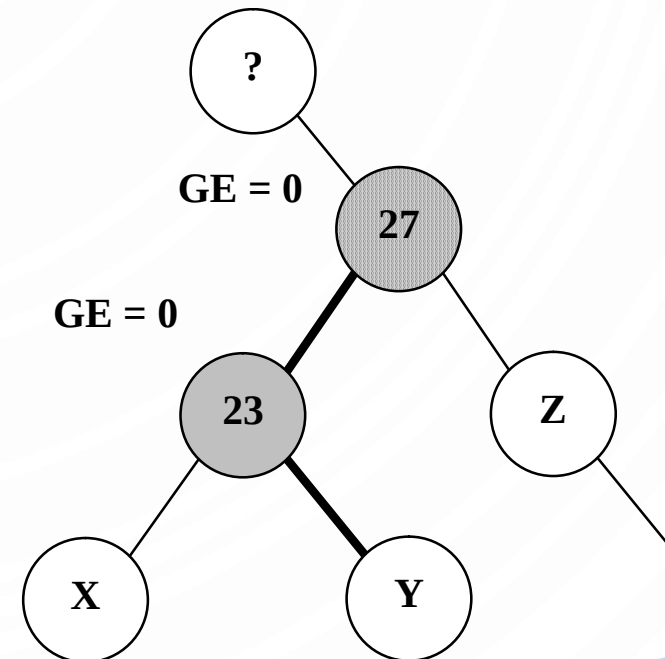
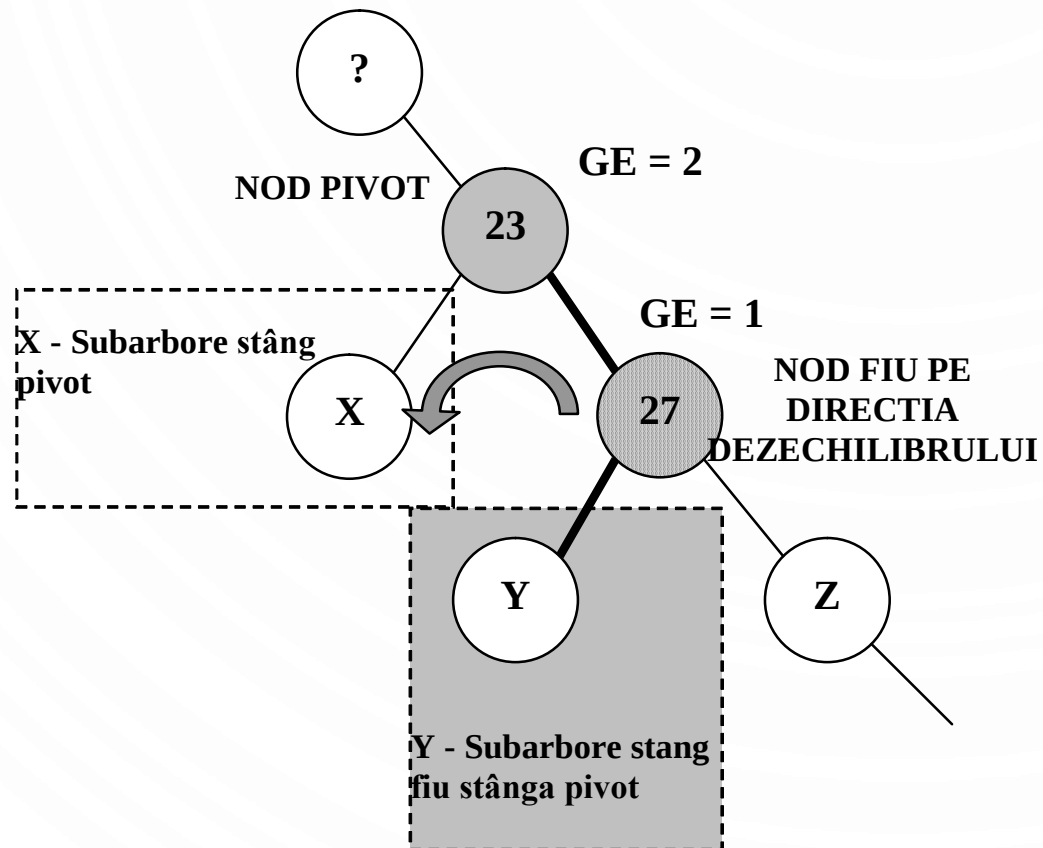
AVL - Operations

Pivot (node 23), has $GE = 2$: unbalanced to the right;
The right child has a minor unbalance to the right as well;
Rebalancing operation: **simple rotation to the left**;



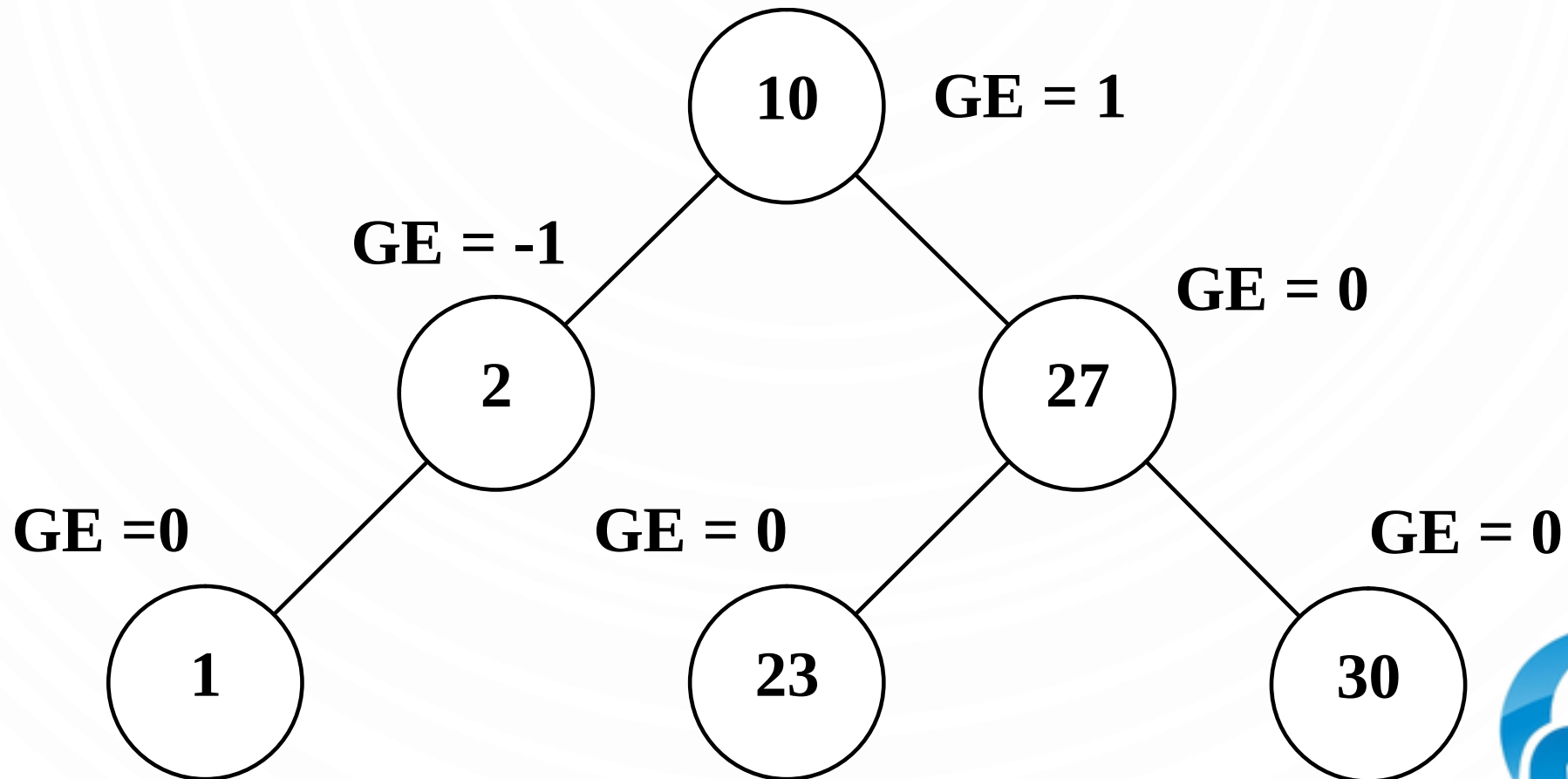
AVL - Operations

Simple rotation to the left



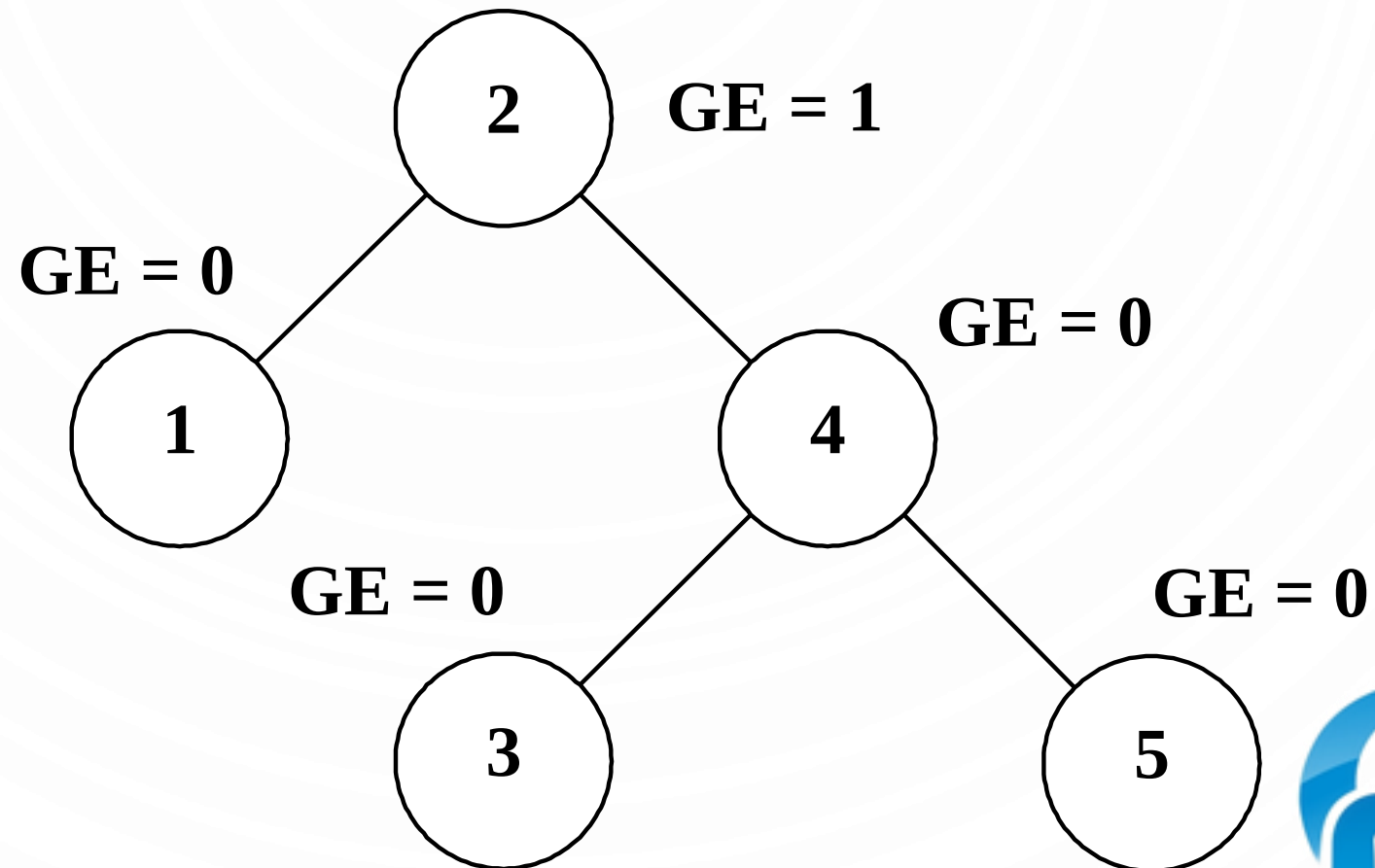
AVL - Operations

Self balanced AVL tree



AVL - Operations

AVL Tree: inserting value 6



AVL - Operations

1. Double rotation: first to the left – second to the right:

- Insertion in the left subtree at the right child level;

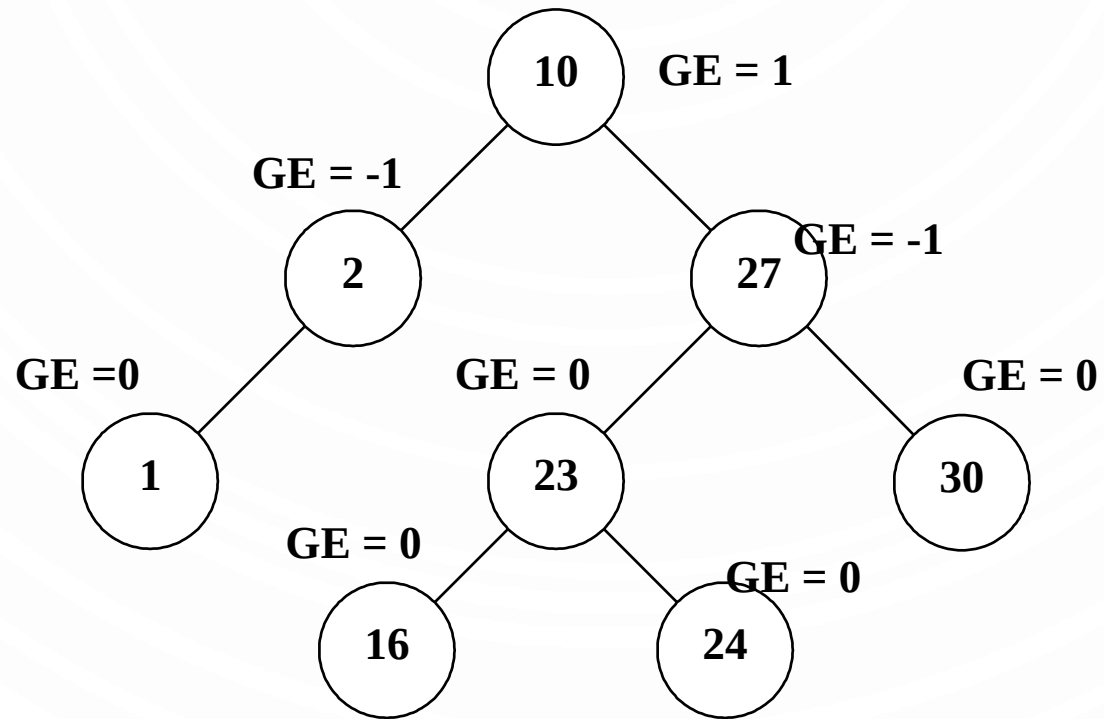
2. Double rotation: first to right – second to the left:

- Insertion in the right subtree at the left child level;



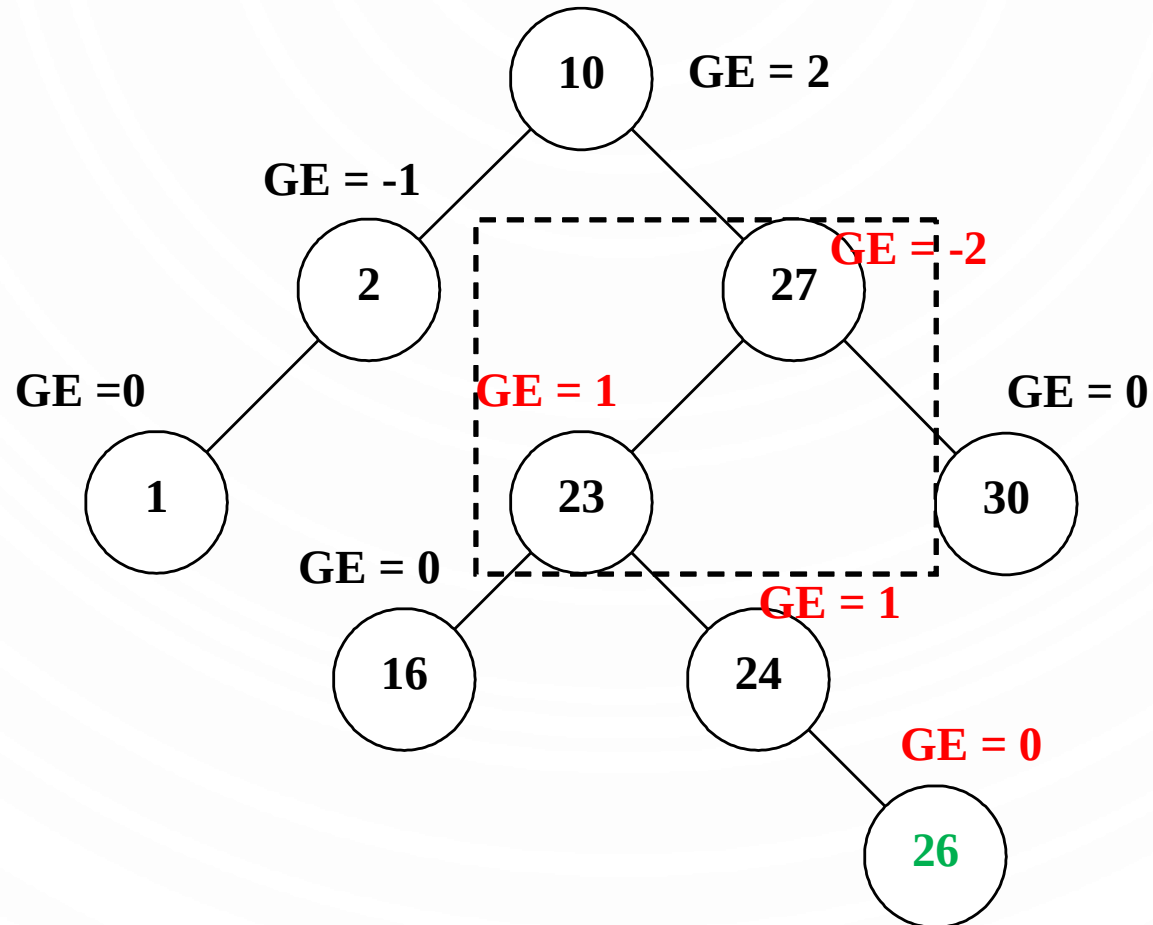
AVL - Operations

A balanced BST



AVL - Operations

Unbalanced AVL tree after inserting the key with value 26;



AVL - Operations

- Last operation: insertion of node with value 26;
- Pivot (node 27), has $GE = -2$: unbalanced to the left;
- The left child (node 23) has a minor unbalance to the right;
- Rebalancing operations:
 - Change the unbalance for the child node to the same order as the parent;
(simple rotation to the left in child node);
 - Simple rotation in the pivot to the opposite direction of the unbalance;
(simple rotation to the right for the pivot);



AVL - Operations

Balancing:

- Applying to consecutive simple rotations:
- **First rotation**: makes that the two nodes (pivot and child) have the same unbalanced direction;
- **Second rotation**: rebalances the tree;



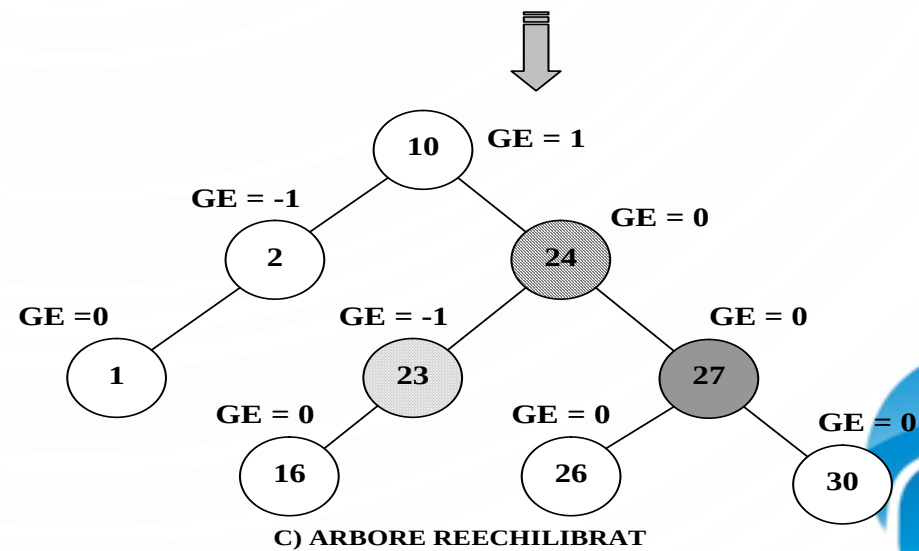
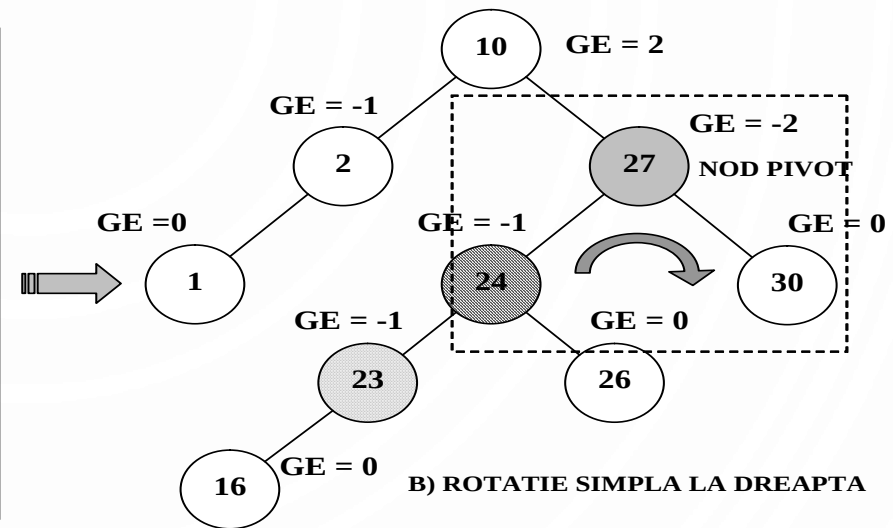
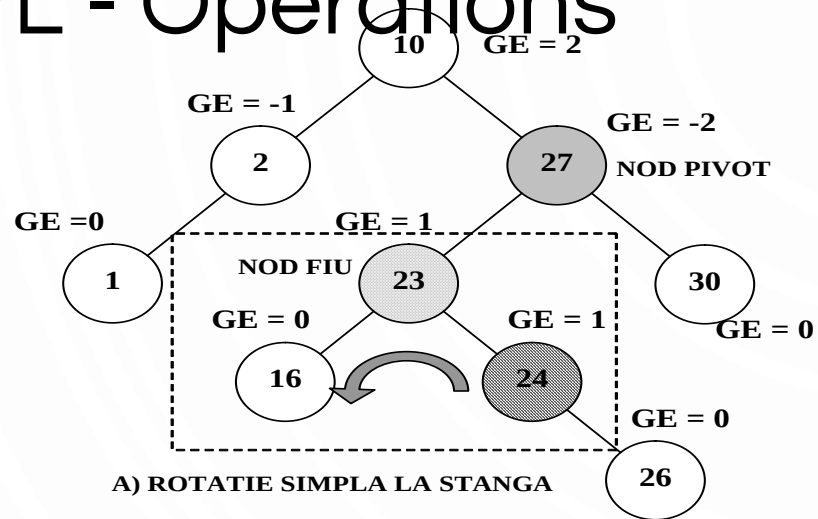
AVL - Operations

Rebalancing a tree: the two rotations are applied to two different nodes:

- First rotation is applied to the child node in the same direction as the imbalance of the parent node, the pivot; this rotation aims to bring the tree in one of the two simple rotations cases;
- The second rotation is applied to the pivot node in the opposite direction of the imbalance in this way rebalancing the tree;

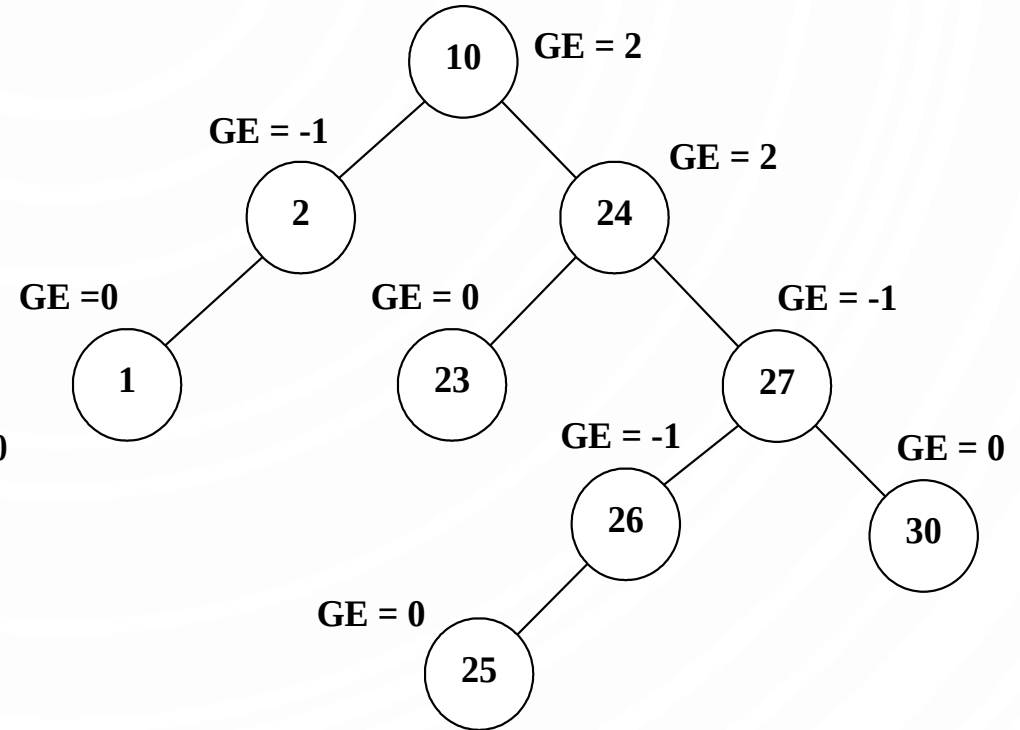
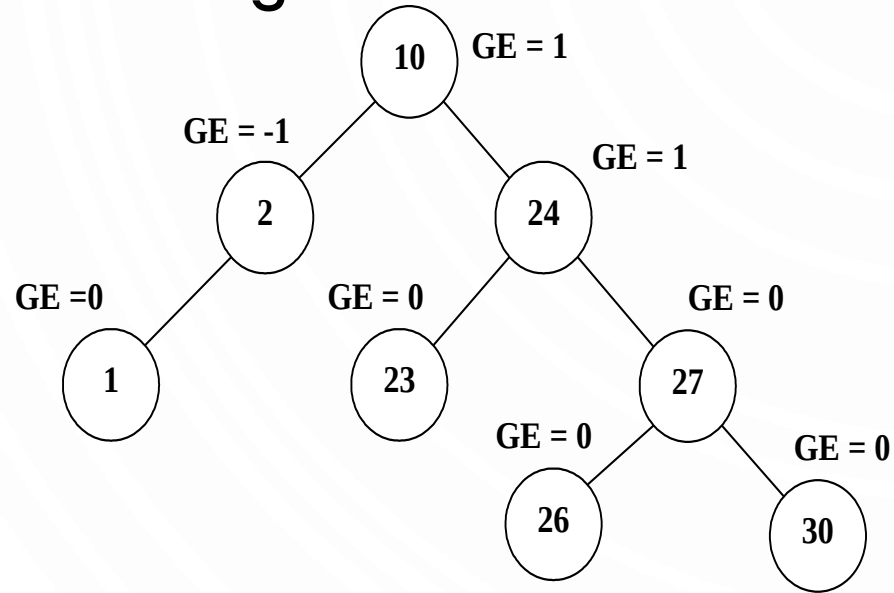


AVL - Operations



AVL - Operations

Inserting the node with value 25:



AVL Trees

There are two node with a strong imbalance factor (24 and 10);

Pivot (node 24) with strong imbalance to the right;

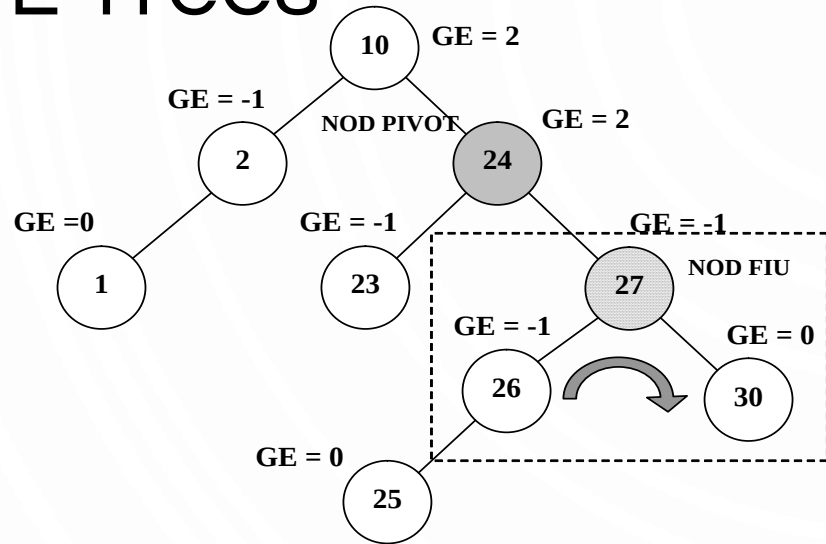
The child from the imbalance direction is node 27 which has a minor imbalance to the left side;

Rebalance operations:

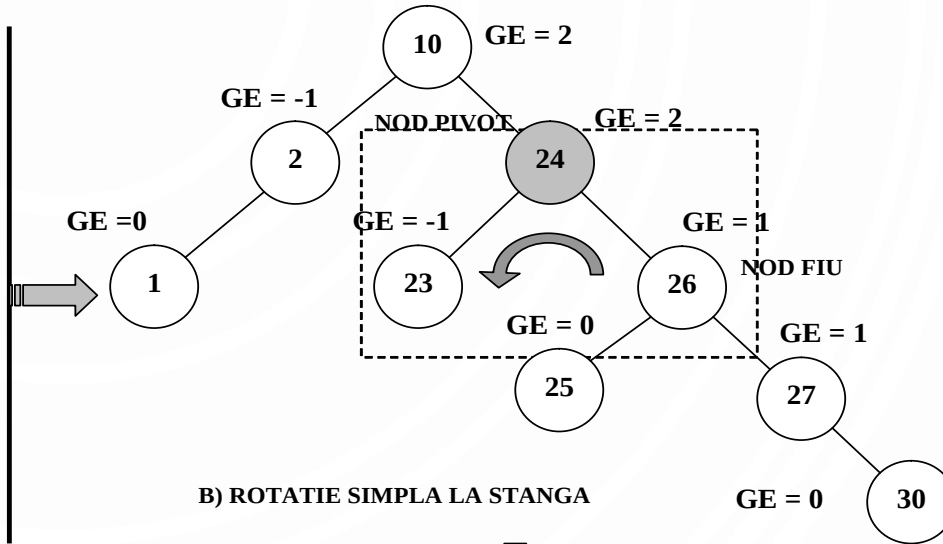
- Simple rotation to the right in 27;
- Simple rotation to the left in 24;



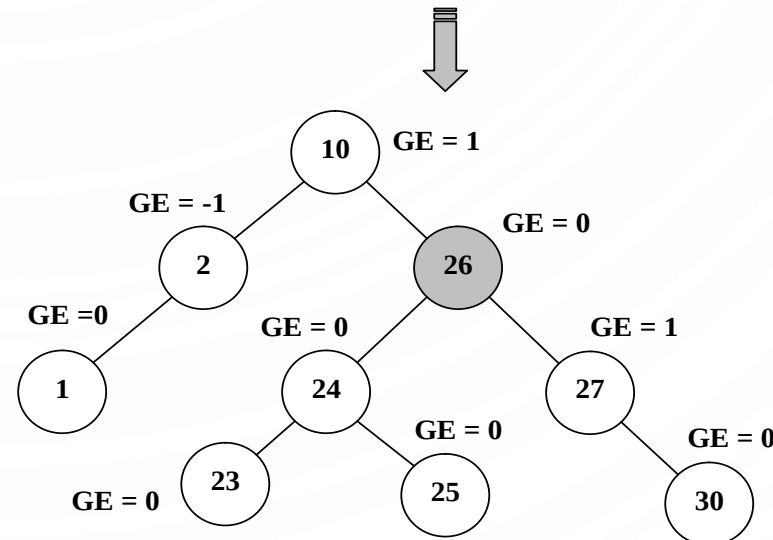
AVL Trees



A) ROTATIE SIMPLA LA DREAPTA



B) ROTATIE SIMPLA LA STANGA



C) ARBORE REECHILIBRAT



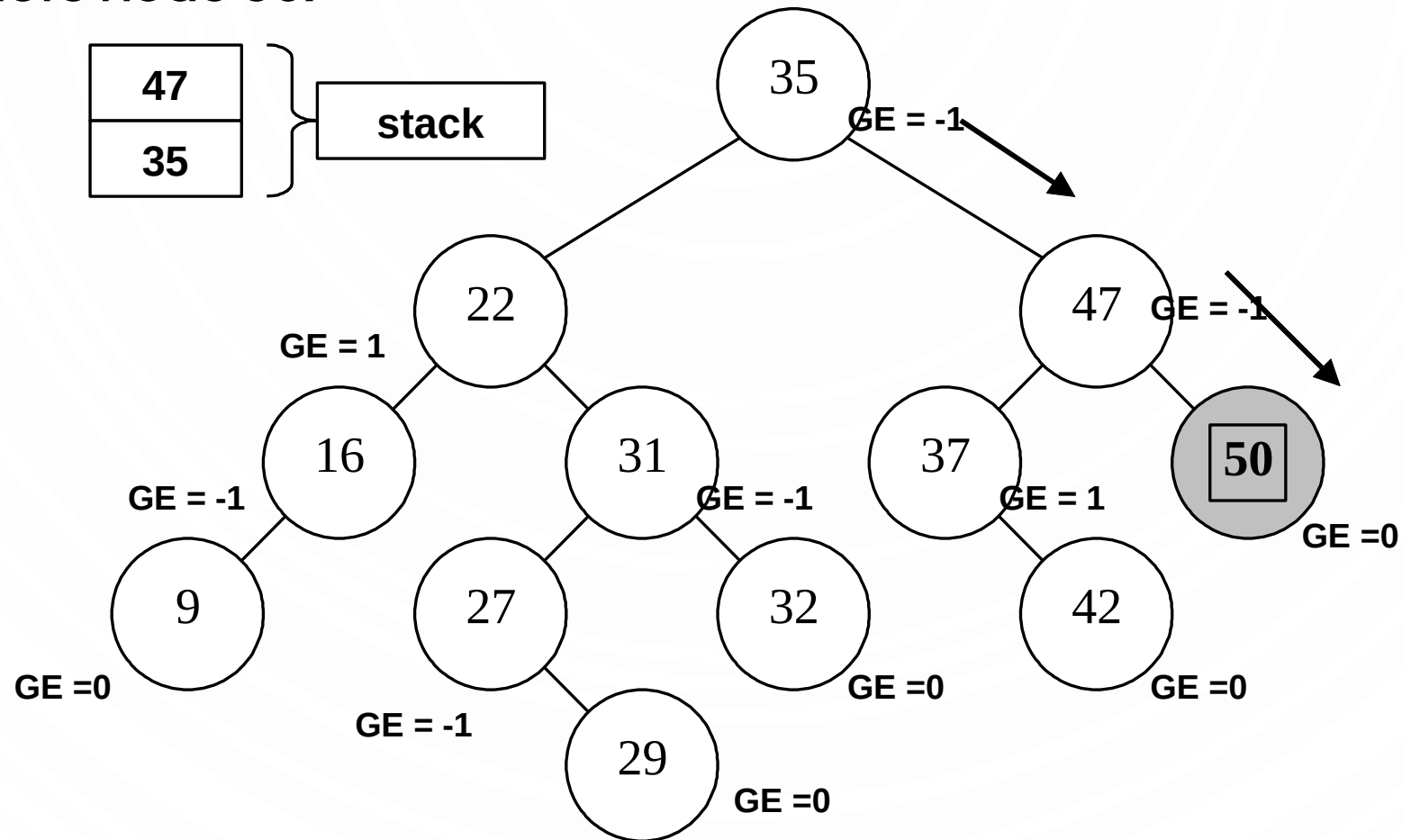
AVL Trees

PIVOT GE values	child	CHILD GE values	Rotations
+2	right	+1	Simple rotation to the left;
+2	right	-1	Double rotation: simple rotation tot the right at child level; simple rotation to the left at pivot level;
-2	left	-1	Simple rotation to the right;
-2	left	+1	Double rotation: simple rotation tot the left at child level; simple rotation to the right at pivot level;

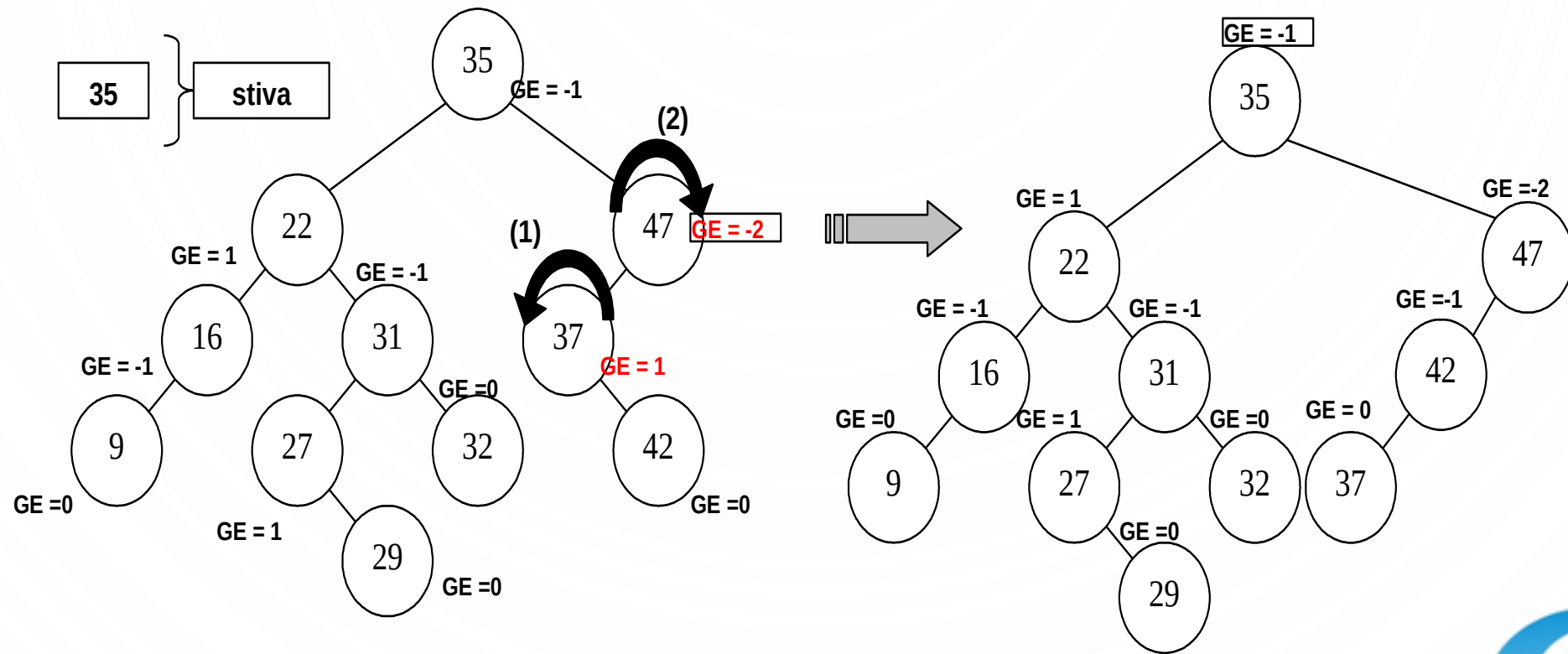


AVL Trees

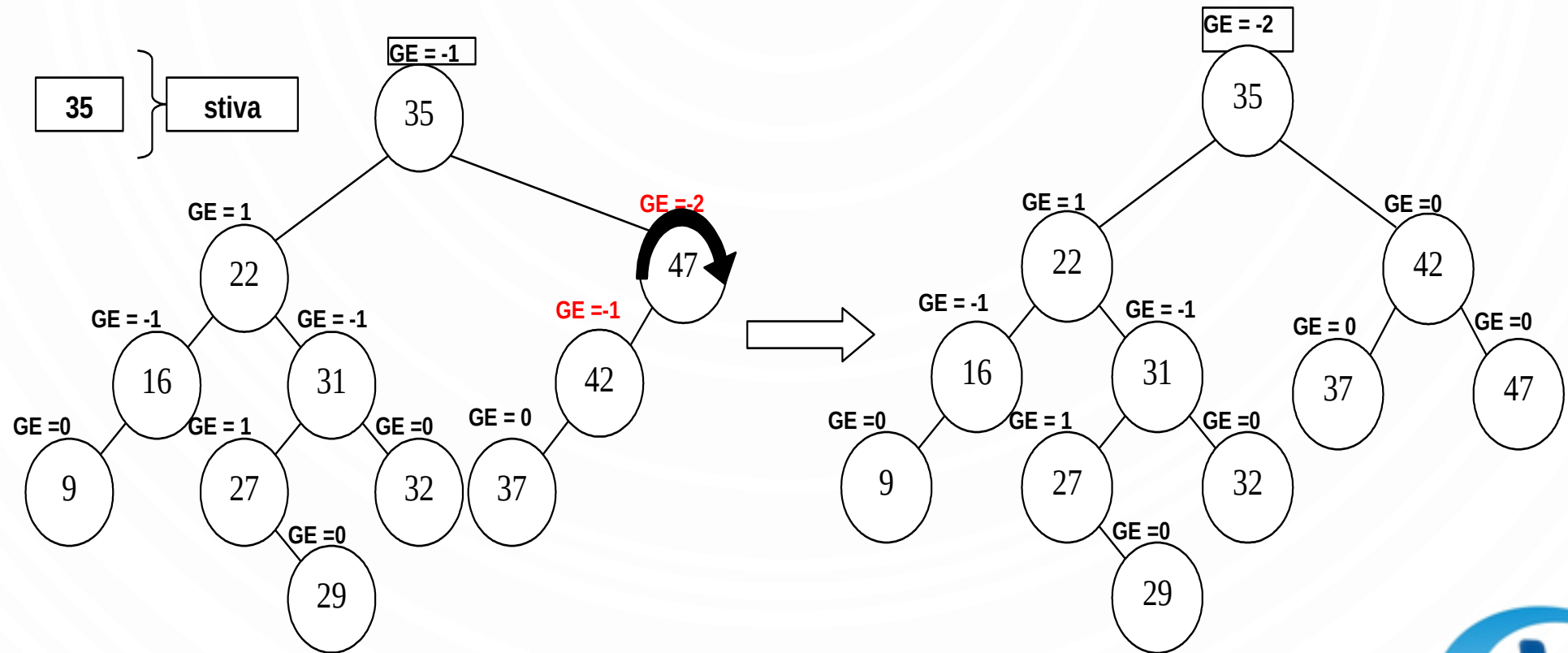
Delete node 50:



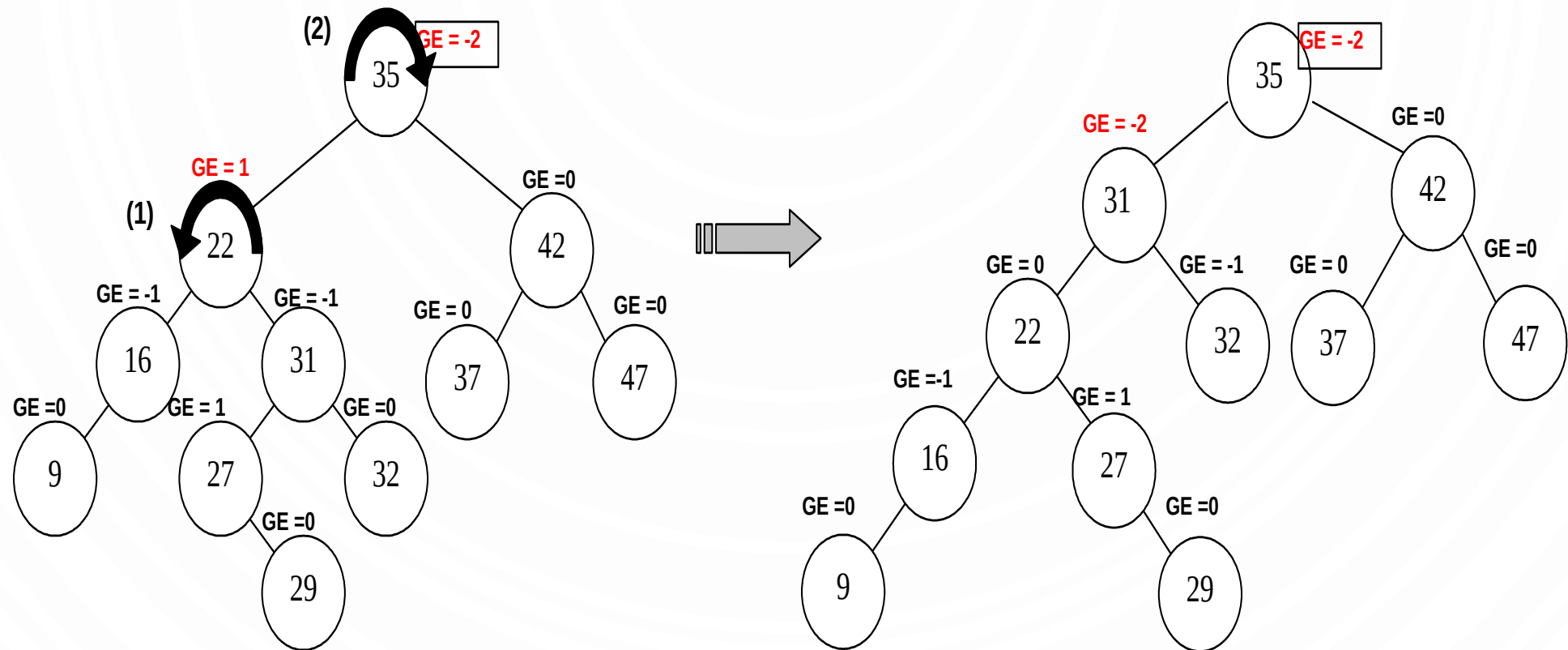
AVL Trees



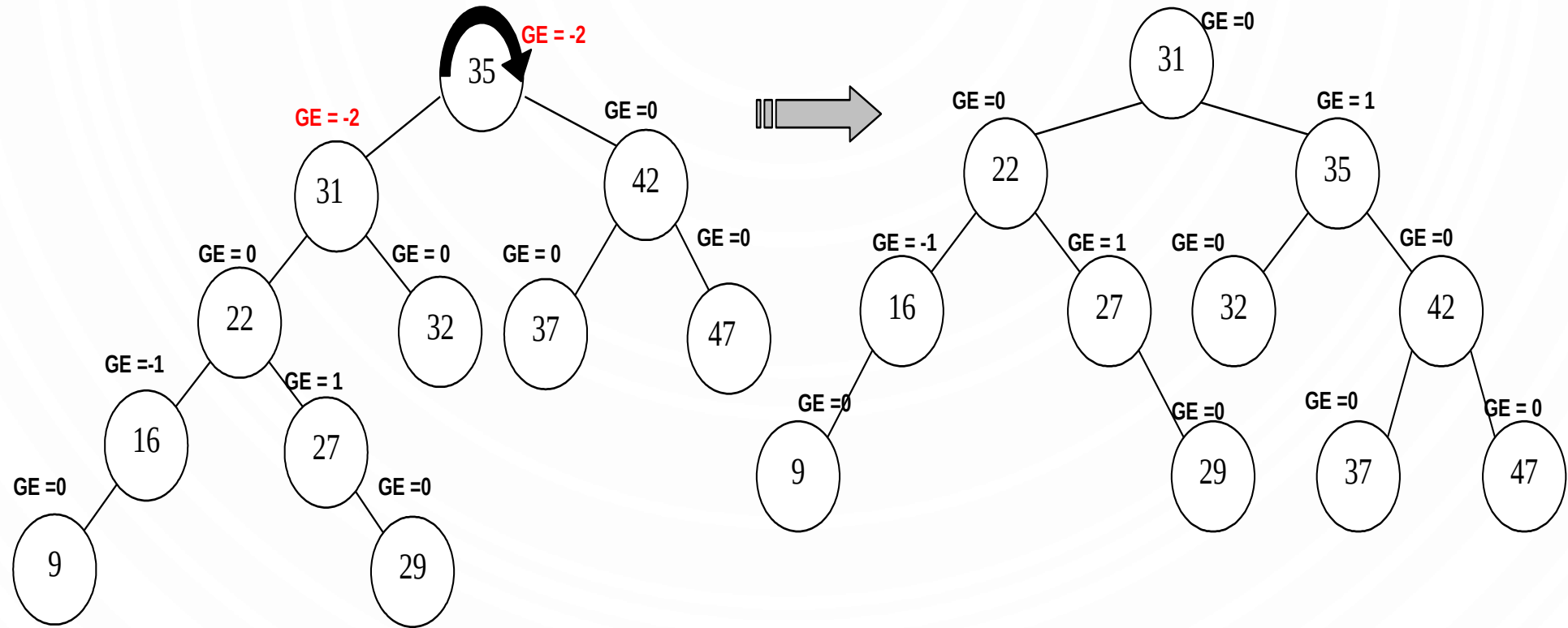
AVL Trees



AVL Trees

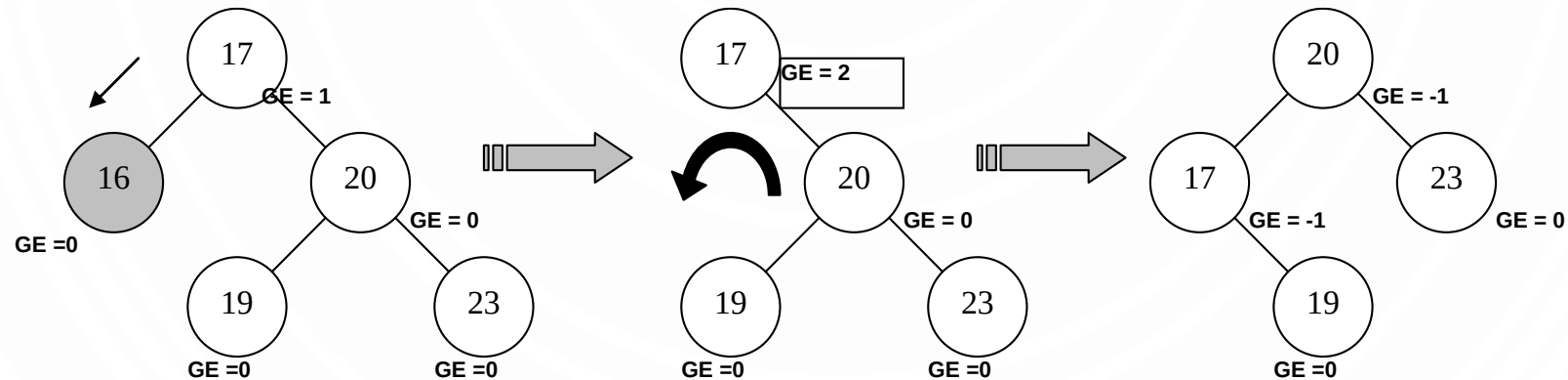


AVL Trees



AVL Trees

Particular unbalanced situation:



After deleting 16, the pivot has an unbalanced factor of +2 and the child is null;

If the **child is null** or has an **imbalance factor of 0**, the opposite child imbalance factor is analyzed in order to determine the correct rotation;

The aforementioned rules are applied. If the imbalance factor is 0 the child is considered as having the same sign.



Types of balanced trees:

- **AVL Trees** Adel'son-Velsii and Landis 1962
- **B-Trees** Bayer and McCreight 1972
- **BB[α] Trees** Nievergelt and Reingold 1973
- **Red-Black Trees**
- **Splay-Trees** Sleator and Tarjan 1985
- **Skip Lists** Pugh 1989
- **Scapegoat Trees** Galperin and Rivest 1993
- **Treaps** Seidel and Aragon 1996



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

